NEXT ENGINEER · _SHEERAT_

Theory of Automata and Formal Languages

BCS402

One-day revision notes: definitions, solved examples, code and important questions

Program: B.Tech | **Branches:** CSE, IT, CSE-AIML, CSE-DS, AIDS | **Semester:** 4 | **Credits:** 4

L-T-P: 3-1-0 | **Scheme:** AKTU B.Tech 2nd year, 2023-24 onwards

UNIT 1 Basic Concepts and Automata Theory

UNIT 2 Regular Expressions and Languages

UNIT 3 Regular and Non-Regular Grammars

UNIT 4 Pushdown Automata and Properties of CFLs

UNIT 5 Turing Machines and Undecidability

Original revision notes written for the AKTU syllabus. Every unit has the same parts:

1. **2-mark definitions**: short answers you can write word for word in Section A.
2. **Core concepts**: theorems, comparison tables and conversion procedures.
3. **Solved examples**: the constructions that always appear (DFA design, NFA to DFA, minimization, Arden's theorem, pumping lemma, CNF, PDA, Turing machine, PCP).
4. **Key points to remember**: the formal definitions and rules you must write correctly.
5. **Important questions**: the long questions to prepare, with what your answer must contain.

How to use these notes the day before the paper

- **Morning (3 hours):** Unit 1. Practise drawing two DFAs, one subset construction and one minimization. These are almost always asked.
- **Afternoon (3 hours):** Units 2 and 3. Arden's theorem, the pumping lemma proof for a^n b^n, ambiguity, simplification and CNF conversion.
- **Evening (2 to 3 hours):** Units 4 and 5. PDA for a^n b^n with an instantaneous-description trace, TM for a^n b^n, the halting problem and PCP, then the "Last-hour checklist".
- **In the exam:** always write the 5-tuple (or 7-tuple) formally, draw the transition diagram, and give the transition table. For a proof, state the lemma first, then choose the string, then show the contradiction.

AKTU paper pattern (theory, 70 marks, 3 hours): Section A has 7 short questions of 2 marks, all compulsory (14). Section B has 5 questions of 7 marks, attempt any 3 (21). Section C has 5 questions (one per unit), each with two parts, attempt one part of each, 7 marks each (35). Every unit gets its own Section C question, so do not skip any unit.

UNIT 1 Basic Concepts and Automata Theory

✍️ 2-mark definitions

- **Alphabet (Σ):** a finite, non-empty set of symbols, e.g. Σ = {0, 1}.
- **String:** a finite sequence of symbols from Σ. The empty string is ε, with |ε| = 0.
- **Σ* and Σ⁺:** Σ* is the set of all strings over Σ including ε. $\Sigma^+ = \Sigma^* - \{\varepsilon\}$.
- **Language:** any subset of Σ*. It may be finite or infinite.
- **Deterministic finite automaton (DFA):** a 5-tuple M = (Q, Σ, δ, q0, F) where δ: Q × Σ → Q gives exactly one next state for each state and input symbol.
- **Nondeterministic finite automaton (NFA):** a 5-tuple where δ: Q × Σ → 2^Q, so a state may have zero, one or many next states on the same symbol.
- **ε-NFA:** an NFA that may also change state without reading input; δ: Q × (Σ ∪ {ε}) → 2^Q.
- **ε-closure(q):** the set of states reachable from q using only ε-moves, including q itself.
- **Extended transition function δ̂:** δ̂(q, ε) = q and δ̂(q, wa) = δ(δ̂(q, w), a). A string w is accepted if δ̂(q0, w) ∈ F.
- **Dead (trap) state:** a non-final state from which no final state can be reached.
- **Moore machine:** a 6-tuple (Q, Σ, Δ, δ, λ, q0) whose output depends only on the present state, λ: Q → Δ.
- **Mealy machine:** a 6-tuple whose output depends on the present state and the present input, λ: Q × Σ → Δ.
- **Equivalent states:** p and q are equivalent if for every string w, δ̂(p, w) and δ̂(q, w) are both final or both non-final.

💡 Core concepts

DFA vs NFA

Point	DFA	NFA
Transition	δ: Q × Σ → Q	δ: Q × Σ → 2^Q
Moves per symbol	exactly one	zero, one or many
ε-moves	not allowed	allowed in an ε-NFA
Design	harder	easier, fewer states
Checking a string	one path	accept if any path ends in F
Power	same	same (every NFA has an equivalent DFA)
States after conversion	n states in NFA can need up to 2^n in DFA	

Equivalence of NFA and DFA (subset construction). Each DFA state is a set of NFA states. Start with {q0}. For a DFA state S and symbol a, the next state is the union of δ(q, a) for every q in S. Any DFA state containing an NFA final state is final. Only build states reachable from the start.

ε-NFA to NFA. Keep the same states. New δ'(q, a) = ε-closure(δ(ε-closure(q), a)). A state is final in the new NFA if its ε-closure contains an old final state.

Moore vs Mealy

Point	Moore	Mealy
Output depends on	present state only	present state and input
Output for input of length n	n + 1 symbols (includes start state)	n symbols
Output written	inside the state	on the transition (a/0)
States needed	usually more	usually fewer
Reaction to input	one step later	immediate

Moore to Mealy: the output on a transition into state q is λ(q). The number of states stays the same.

Mealy to Moore: if a state q is entered with different outputs, split it into one copy per output (q0 with output 0, q1 with output 1). Each copy keeps q's outgoing transitions.

Minimization of a DFA (partition / equivalence method).

1. Remove states not reachable from the start state.
2. P0 = {F, Q − F}.
3. Build P(k+1) from Pk: two states stay in the same group only if, for every input symbol, they go to states in the same group of Pk.
4. Stop when P(k+1) = Pk. Each group becomes one state.

Myhill–Nerode table-filling is an equivalent method: mark pairs (final, non-final) first, then mark a pair if some symbol takes it to a marked pair.

🧮 Solved examples

Example 1 (DFA for strings ending in "ab", Σ = {a, b}). States remember how much of "ab" has just been seen.

State	a	b
→q0	q1	q0
q1	q1	q2
*q2	q1	q0

Check "aab": q0 →a q1 →a q1 →b q2, final, accepted.

Example 2 (DFA for binary numbers divisible by 3). Reading bit b changes the remainder r to (2r + b) mod 3.

State (remainder)	0	1
→*r0	r0	r1
r1	r2	r0

NEXT ENGINEER _sheerat_

State (remainder)	0	1
r2	r1	r2

Check 110 (= 6): r0 →1 r1 →1 r0 →0 r0, accepted.

Example 3 (NFA to DFA). NFA for strings over {0, 1} ending in "01": δ(q0, 0) = {q0, q1}, δ(q0, 1) = {q0}, δ(q1, 1) = {q2}, final q2.

DFA state	0	1
→{q0}	{q0, q1}	{q0}
{q0, q1}	{q0, q1}	{q0, q2}
*{q0, q2}	{q0, q1}	{q0}

Only 3 of the 8 possible subsets are reachable.

Example 4 (minimization). DFA over {0, 1}, start A, final E: A(0→B, 1→C), B(0→B, 1→D), C(0→B, 1→C), D(0→B, 1→E), E(0→B, 1→C).

- P0 = {A, B, C, D}, {E}.
- On 1, D goes to E (a different group), the others stay. P1 = {A, B, C}, {D}, {E}.
- On 1, B goes to D, while A and C go to C. P2 = {A, C}, {B}, {D}, {E}.
- A and C both go to B on 0 and C on 1. P3 = P2, so stop.

The minimal DFA has 4 states: [AC], B, D, E. It accepts strings ending in "011".

Example 5 (Mealy to Moore). Mealy machine that outputs 1 when "ab" is seen: q0 (a/0 → q1, b/0 → q0), q1 (a/0 → q1, b/1 → q0). q0 is entered with outputs 0 and 1, so split it.

Moore state	a	b	Output
→q0/0	q1/0	q0/0	0
q0/1	q1/0	q0/0	1
q1/0	q1/0	q0/1	0

Key points to remember

- Always write the formal tuple: M = (Q, Σ, δ, q0, F).
- A DFA must have a transition for every symbol from every state. Add a dead state if needed.
- Number of DFA states after subset construction is at most 2^n.
- Minimization never changes the language. The minimal DFA is unique up to renaming of states.
- Moore output length is n + 1; Mealy output length is n.

⭐ Important questions

1. Define DFA and NFA formally. Prove that for every NFA there is an equivalent DFA, with an example.

2. Design a DFA for a given language (ending with, containing, divisible by k, even number of a's and b's).
3. Convert an ε-NFA to an NFA and then to a DFA.
4. Minimize a given DFA using the partition method; show every Pk.
5. Compare Moore and Mealy machines. Convert one into the other with an example.

NEXT ENGINEER
sheerat

UNIT 2 Regular Expressions and Languages

✍️ 2-mark definitions

- **Regular expression (RE):** ø, ε and each a ∈ Σ are REs. If r and s are REs, then r + s, rs and r* are REs. Nothing else is an RE.
- **Regular language:** a language that can be described by an RE, or equivalently accepted by a finite automaton.
- **Kleene closure r*:** zero or more concatenations of r. r^+ = r r* is one or more.
- **Transition graph:** a generalized automaton whose edges are labelled by strings or REs rather than single symbols.
- **Kleene's theorem:** a language is regular if and only if it is accepted by some finite automaton; equivalently, FA and REs describe the same class of languages.
- **Arden's theorem:** if P and Q are REs and P does not contain ε, then the equation R = Q + RP has the unique solution R = QP*.
- **Non-regular language:** a language no finite automaton accepts, e.g. $\{a^n b^n \mid n \geq 0\}$.
- **Closure property:** an operation under which the result of applying it to regular languages is always regular.
- **Decision property:** a question about regular languages (emptiness, finiteness, membership, equivalence) for which an algorithm always gives the answer.

💡 Core concepts

Useful RE identities

Identity	Identity
ø + R = R	øR = Rø = ø
εR = Rε = R	ε* = ε, ø* = ε
R + R = R	R*R* = R*
(R*)* = R*	RR* = R*R = R^+
ε + RR* = R*	(P + Q)* = (P*Q*)* = (P* + Q*)*
(PQ)*P = P(QP)*	R(P + Q) = RP + RQ

RE to ε-NFA (Thompson construction). Build small machines and join them.

- a: start →a→ final.
- r + s: a new start with ε-moves to the starts of r and s, and ε-moves from their finals to a new final.
- rs: ε-move from the final of r to the start of s.
- r*: a new start and final, with ε-moves start → r-start, r-final → r-start, r-final → final, and start → final.

FA to RE. Two methods:

1. **Arden's method:** write one equation per state, qi = sum of (qj · a) for every edge qj →a→ qi, adding ε for the start state. Solve with Arden's theorem and substitution. The RE is the sum of the solutions for the final states.
2. **State elimination:** remove states one at a time, replacing paths p → q → r by the RE (p→q)(q→q)* (q→r).

Pumping lemma for regular languages. If L is regular, there is a constant n such that every w in L with $|w| \geq n$ can be written w = xyz with:

1. $|xy| \leq n$,
2. $|y| \geq 1$,
3. $x\, y^k\, z \in L$ for every $k \geq 0$.

It is used only to prove a language is **not** regular, by contradiction.

Closure properties of regular languages. Union, concatenation, Kleene star, complement, intersection, difference, reversal, homomorphism and inverse homomorphism all give regular languages. For complement, swap final and non-final states of a complete DFA. For intersection, use the product construction with states (p, q).

Decision properties.

- Emptiness: is any final state reachable from the start?
- Finiteness: is there a cycle on some path from the start to a final state?
- Membership: run the DFA on w.
- Equivalence: minimize both DFAs and compare, or check that (L1 − L2) ∪ (L2 − L1) is empty.

🧮 Solved examples

Example 1 (writing REs).

- Strings over {a, b} containing "aa": (a + b)* aa (a + b)*.
- Strings starting and ending with different symbols: a(a + b)*b + b(a + b)*a.
- Strings with an even number of a's (any b's): (b* a b* a b*)* + b*, which simplifies to (b + ab*a)*.
- Binary strings with no two consecutive 1s: (0 + 10)*(ε + 1).

Example 2 (Arden's theorem). DFA: q1 is start and final, q1 →a→ q2, q2 →a→ q2, q2 →b→ q1.

Equations:

- q1 = ε + q2 b
- q2 = q1 a + q2 a

Apply Arden to q2 with R = q2, Q = q1 a, P = a: q2 = q1 a a*.

Substitute: q1 = ε + q1 a a* b. Apply Arden with Q = ε, P = a a* b: q1 = ε(aa*b)* = (aa*b)* = $(a^+b)^*$.

Since q1 is the only final state, the RE is **$(a^+b)^*$**.

Example 3 (pumping lemma: L = {$a^n b^n$ | n ≥ 0} is not regular).

1. Assume L is regular, with pumping constant n.
2. Choose w = $a^n b^n$. Then w ∈ L and $|w| = 2n \geq n$.
3. Since $|xy| \leq n$, x and y contain only a's. Let y = a^m with m ≥ 1.
4. Pump with k = 2: $x\, y^2\, z = a^{n+m}\, b^n$. Here n + m ≠ n, so this string is not in L.

NEXT ENGINEER sheerat_

5. This contradicts the lemma, so L is not regular.

Example 4 (L = {a^(i²) | i ≥ 1} is not regular). Choose w = a^(n²). Then y = a^m with 1 ≤ m ≤ n. Pumping with k = 2 gives length $n^2 + m$, and $n^2 < n^2 + m \le n^2 + n < (n + 1)^2$. This length lies strictly between two consecutive perfect squares, so the string is not in L, a contradiction.

Example 5 (product construction). For the intersection of "even number of a's" (states E, O) and "ends in b" (states X, Y, final Y), the product DFA has states (E, X), (E, Y), (O, X), (O, Y). The only final state is (E, Y).

Key points to remember

- Arden's theorem needs P to not contain ε; otherwise the solution is not unique.
- In the pumping lemma, you choose w, but you must handle **every** possible split xyz allowed by |xy| ≤ n and |y| ≥ 1.
- Every finite language is regular.
- A regular language can have a non-regular subset: a*b* is regular, but $a^n b^n$ inside it is not.
- Complement needs a complete DFA (add the dead state first).

⭐ Important questions

1. State and prove Arden's theorem. Use it to find the RE of a given DFA.
2. State Kleene's theorem. Construct an ε-NFA for a given RE, e.g. (a + b)*abb.
3. State the pumping lemma for regular languages and prove that $a^n b^n$ (or ww, or a^p for prime p) is not regular.
4. Explain the closure properties of regular languages with proofs for union, complement and intersection.
5. Explain the decision properties of regular languages.

UNIT 3 Regular and Non-Regular Grammars

✍️ 2-mark definitions

- **Grammar:** a 4-tuple G = (V, T, P, S) of variables, terminals, productions and a start symbol.
- **Context-free grammar (CFG):** every production has the form A → α, where A is a single variable and α ∈ (V ∪ T)*.
- **Derivation:** a sequence of production applications from S to a string of terminals. Leftmost derivations always replace the leftmost variable; rightmost derivations replace the rightmost.
- **Derivation (parse) tree:** a tree whose root is S, internal nodes are variables, leaves are terminals or ε, and each node's children match a production.
- **Ambiguous grammar:** a grammar in which some string has two or more different parse trees (or two leftmost derivations).
- **Inherently ambiguous language:** a CFL for which every grammar is ambiguous, e.g. $\{a^i b^j c^k \mid i = j \text{ or } j = k\}$.
- **Right-linear grammar:** every production is A → wB or A → w, with w ∈ T*. **Left-linear:** A → Bw or A → w. Both generate exactly the regular languages.
- **Useless symbol:** a symbol that is not generating (cannot derive a terminal string) or not reachable from S.
- **Unit production:** a production A → B with one variable on each side.
- **ε-production / nullable variable:** A → ε; a variable is nullable if it can derive ε.
- **Chomsky Normal Form (CNF):** every production is A → BC or A → a.
- **Greibach Normal Form (GNF):** every production is A → aα, where a is a terminal and α is a string of variables.

💡 Core concepts

Chomsky hierarchy

Type	Grammar	Production rule	Language	Machine
0	Unrestricted	α → β, α ≠ ε	Recursively enumerable	Turing machine
1	Context-sensitive	α → β with |α| ≤ |β|	Context-sensitive	Linear bounded automaton
2	Context-free	A → α	Context-free	Pushdown automaton
3	Regular	A → aB or A → a	Regular	Finite automaton

Each class is a proper subset of the one above it: Type 3 ⊂ Type 2 ⊂ Type 1 ⊂ Type 0.

FA to right-linear grammar. For each transition δ(p, a) = q write p → aq. For each final state q write q → ε. The start symbol is the start state.

Right-linear grammar to FA. Each variable is a state. A → aB becomes a transition A →a→ B. A → a becomes a transition from A to a new final state.

Simplification of a CFG (in this order).

1. Remove ε-productions: find nullable variables; for each production, add every version with nullable variables left out; delete A → ε (keep S → ε if ε ∈ L).
2. Remove unit productions: for each unit pair (A, B) with A ⇒* B, add A → α for every non-unit B → α.
3. Remove useless symbols: first remove non-generating symbols, then non-reachable ones. The order matters.

Conversion to CNF.

1. Simplify the grammar (steps above).
2. For each terminal a appearing in a body of length ≥ 2, add C_a → a and replace a by C_a.
3. Break bodies longer than 2: A → B1 B2 B3 becomes A → B1 D, D → B2 B3.

Conversion to GNF (outline). Convert to CNF, number the variables A1 ... An, make every production Ai → Aj... have j > i by substitution, remove left recursion with new variables (A → Aα | β becomes A → β | βZ, Z → α | αZ), then substitute back so that each body starts with a terminal.

🧮 Solved examples

Example 1 (CFGs for common languages).

- $\{a^n b^n \mid n \geq 0\}$: S → aSb | ε.
- Palindromes over {a, b}: S → aSa | bSb | a | b | ε.
- Equal number of a's and b's: S → aSbS | bSaS | ε.
- $\{a^n b^{2n} \mid n \geq 1\}$: S → aSbb | abb.
- Balanced parentheses: S → (S) | SS | ε.

Example 2 (ambiguity). Grammar E → E + E | E * E | id. The string id + id * id has two leftmost derivations:

1. E ⇒ E + E ⇒ id + E ⇒ id + E * E ⇒ id + id * E ⇒ id + id * id (multiplication done first).
2. E ⇒ E * E ⇒ E + E * E ⇒ id + E * E ⇒ id + id * E ⇒ id + id * id (addition done first).

Two parse trees exist, so the grammar is ambiguous. The unambiguous version uses precedence levels: E → E + T | T, T → T * F | F, F → id | (E).

Example 3 (simplification). Grammar: S → aA | aBB, A → aAA | ε, B → bB | bbC, C → B.

- Nullable: A. Remove A → ε: S → aA | a | aBB, A → aAA | aA | a.
- Unit production C → B: C → bB | bbC.
- Generating: A (A → a), S (S → a). B and C never reach a terminal string, since every B and C production contains B or C. So B and C are useless; remove them and any production using them.
- Result: S → aA | a, A → aAA | aA | a.

Example 4 (CNF). Convert S → aSb | ab.

1. Add A → a and B → b: S → ASB | AB.
2. Break ASB: S → AC, C → SB.

CNF: S → AC | AB, C → SB, A → a, B → b.

Example 5 (GNF, short case). S → AB | a, A → aA | b, B → b. Substitute A in S → AB: S → aAB | bB | a. Now every production starts with a terminal followed only by variables: S → aAB | bB | a, A → aA |

b, B → b. This is GNF.

Example 6 (FA to grammar). DFA for strings ending in "ab" (Unit 1, Example 1): q0 → aq1 | bq0, q1 → aq1 | bq2, q2 → aq1 | bq0 | ε.

Key points to remember

- To prove ambiguity, show two parse trees or two leftmost derivations for **one** string.
- Simplify in the order: ε-productions, unit productions, useless symbols (non-generating, then non-reachable).
- In CNF, a string of length n has a derivation of exactly 2n − 1 steps.
- In GNF, a string of length n has a derivation of exactly n steps.
- Every regular language is context-free, but not every CFL is regular.

⭐ Important questions

1. Define CFG, derivation tree and ambiguity. Show that a given grammar is ambiguous and remove the ambiguity.
2. Explain the Chomsky hierarchy with grammars, languages and machines.
3. Simplify a given CFG (remove ε, unit and useless productions).
4. Convert a given CFG to CNF and to GNF.
5. Construct a regular grammar for a given FA, and an FA for a given regular grammar.

UNIT 4 Pushdown Automata and Properties of CFLs

✍️ 2-mark definitions

- **Pushdown automaton (PDA):** a 7-tuple $P = (Q, \Sigma, \Gamma, \delta, q_0, Z_0, F)$, where Γ is the stack alphabet, Z_0 the initial stack symbol and $\delta: Q \times (\Sigma \cup \{\varepsilon\}) \times \Gamma \to$ finite subsets of $Q \times \Gamma^*$.
- **Instantaneous description (ID):** a triple (q, w, γ) giving the current state, the remaining input and the stack contents (top on the left). $\vdash$ shows one move.
- **Acceptance by final state:** $L(P) = \{w \mid (q_0, w, Z_0) \vdash^* (q, \varepsilon, \gamma), q \in F\}$.
- **Acceptance by empty stack:** $N(P) = \{w \mid (q_0, w, Z_0) \vdash^* (q, \varepsilon, \varepsilon)\}$. Both methods accept the same class of languages.
- **Deterministic PDA (DPDA):** a PDA with at most one move in any situation, and no ε-move where an input move is also possible.
- **Deterministic CFL (DCFL):** a language accepted by a DPDA. Every DCFL is unambiguous.
- **Two-stack PDA:** a PDA with two stacks; it is as powerful as a Turing machine.
- **Pumping lemma for CFLs:** if L is context-free, there is n such that every $z \in L$ with $|z| \geq n$ can be written $uvwxy$ with $|vwx| \leq n$, $|vx| \geq 1$ and $u\, v^k\, w\, x^k\, y \in L$ for all $k \geq 0$.

💡 Core concepts

NPDA vs DPDA

Point	NPDA	DPDA
Moves	may have several choices	at most one choice
Power	accepts all CFLs	accepts only DCFLs
Example	ww^R (even palindromes)	wcw^R (centre marked)
Relation		DCFL ⊂ CFL (proper)
Ambiguity	language may be ambiguous	language always has an unambiguous grammar

Unlike finite automata, nondeterminism adds power to PDAs.

CFG to PDA (single state, empty stack).

- Push S first: start with S on the stack.
- For each production $A \to \alpha$: $\delta(q, \varepsilon, A)$ contains (q, α).
- For each terminal a: $\delta(q, a, a) = \{(q, \varepsilon)\}$.

PDA to CFG (outline). For a PDA accepting by empty stack, create variables $[p\ X\ q]$ meaning "starting in p with X on top, the PDA can pop X and end in q". $S \to [q_0\ Z_0\ p]$ for every p. For $\delta(q, a, X)$ containing $(r, Y_1 \ldots Y_k)$, add $[q\ X\ s_k] \to a\ [r\ Y_1\ s_1][s_1\ Y_2\ s_2] \ldots [s_{(k-1)}\ Y_k\ s_k]$ for all choices of states $s_1 \ldots s_k$. When $k = 0$, add $[q\ X\ r] \to a$.

Closure properties of CFLs. CFLs are closed under union, concatenation, Kleene star, reversal, homomorphism and intersection with a regular language. They are **not** closed under intersection or

complement. Example: L1 = {$a^n b^n c^m$} and L2 = {$a^m b^n c^n$} are CFLs, but L1 ∩ L2 = {$a^n b^n c^n$} is not.

Decision properties of CFLs. Emptiness, finiteness and membership (CYK algorithm, $O(n^3)$) are decidable. Equivalence of two CFGs, ambiguity of a CFG and whether a CFL is regular are undecidable.

Closure summary for all classes

Operation	Regular	CFL	DCFL	Recursive	RE
Union	Yes	Yes	No	Yes	Yes
Intersection	Yes	No	No	Yes	Yes
Complement	Yes	No	Yes	Yes	No
Concatenation	Yes	Yes	No	Yes	Yes
Kleene star	Yes	Yes	No	Yes	Yes
Intersection with regular	Yes	Yes	Yes	Yes	Yes
Reversal	Yes	Yes	No	Yes	Yes

🧮 Solved examples

Example 1 (PDA for L = {$a^n b^n$ | n ≥ 1}, final state). Push each a; pop one a for each b.

- δ(q0, a, Z0) = (q0, aZ0)
- δ(q0, a, a) = (q0, aa)
- δ(q0, b, a) = (q1, ε)
- δ(q1, b, a) = (q1, ε)
- δ(q1, ε, Z0) = (qf, Z0)

Trace for aabb: (q0, aabb, Z0) ⊢ (q0, abb, aZ0) ⊢ (q0, bb, aaZ0) ⊢ (q1, b, aZ0) ⊢ (q1, ε, Z0) ⊢ (qf, ε, Z0). Accepted.

Trace for aab: ... ⊢ (q1, ε, aZ0). No move is possible and qf is not reached, so it is rejected.

Example 2 (NPDA for even palindromes L = {ww^R | w ∈ {a, b}*}). In q0, push every symbol, but at any point guess the middle with an ε-move to q1. In q1, pop when the input matches the top.

- δ(q0, a, X) = (q0, aX) and δ(q0, b, X) = (q0, bX) for X ∈ {a, b, Z0}
- δ(q0, ε, X) = (q1, X)
- δ(q1, a, a) = (q1, ε), δ(q1, b, b) = (q1, ε)
- δ(q1, ε, Z0) = (qf, Z0)

The guess makes it nondeterministic; no DPDA can accept this language.

Example 3 (CFG to PDA). Grammar S → aSb | ab.

- δ(q, ε, S) = {(q, aSb), (q, ab)}
- δ(q, a, a) = {(q, ε)}, δ(q, b, b) = {(q, ε)}

Trace for aabb: (q, aabb, S) ⊢ (q, aabb, aSb) ⊢ (q, abb, Sb) ⊢ (q, abb, abb) ⊢ (q, bb, bb) ⊢ (q, b, b) ⊢ (q, ε, ε). Empty stack, so accepted.

Example 4 (pumping lemma: L = {$a^n b^n c^n$ | n ≥ 1} is not context-free).

NEXT ENGINEER sheerat_

1. Assume L is a CFL with constant n. Choose z = $a^n b^n c^n$.
2. Since $|vwx| \le n$, the part vwx can touch at most two of the three blocks (it cannot contain both a's and c's).
3. Pump with k = 2. At most two of the letter counts increase, and at least one (since $|vx| \ge 1$) does.
4. The three counts are no longer equal, so $u\, v^2\, w\, x^2\, y \notin L$. This is a contradiction, so L is not a CFL.

The same idea proves {ww | w ∈ {a, b}*} and {a^(n²)} are not context-free.

Key points to remember

- The PDA's 7-tuple: (Q, Σ, Γ, δ, q0, Z0, F). Always write δ in the form δ(state, input, top) = (state, push string).
- Writing ε as the push string means pop. Writing X back means leave the stack unchanged.
- Final-state and empty-stack acceptance are equivalent for NPDAs (not for DPDAs).
- Regular ⊂ DCFL ⊂ CFL.
- A two-stack PDA equals a Turing machine in power.

⭐ Important questions

1. Define a PDA formally. Design a PDA for $a^n b^n$ (or $a^n b^{2n}$, or equal a's and b's) and show the ID trace for a string.
2. Prove that acceptance by final state and by empty stack are equivalent.
3. Convert a given CFG to a PDA, or a given PDA to a CFG.
4. State the pumping lemma for CFLs and prove $a^n b^n c^n$ is not context-free.
5. Explain closure and decision properties of CFLs; show that CFLs are not closed under intersection and complement.

UNIT 5 Turing Machines and Undecidability

✍️ 2-mark definitions

- **Turing machine (TM):** a 7-tuple $M = (Q, \Sigma, \Gamma, \delta, q_0, B, F)$, with an infinite tape, a read/write head and $\delta: Q \times \Gamma \to Q \times \Gamma \times \{L, R\}$. B is the blank symbol.
- **ID of a TM:** $\alpha\, q\, \beta$, meaning the tape holds $\alpha\beta$, the machine is in state q and the head is on the first symbol of β.
- **Recursively enumerable (RE) language:** a language accepted by some TM. On strings not in the language the TM may run forever.
- **Recursive (decidable) language:** a language accepted by a TM that halts on every input.
- **Linear bounded automaton (LBA):** a nondeterministic TM whose head never leaves the part of the tape holding the input (between end markers). LBAs accept exactly the context-sensitive languages.
- **Universal Turing machine (UTM):** a TM that takes an encoding $\langle M, w \rangle$ of any TM M and input w and simulates M on w.
- **Church–Turing thesis:** any function that can be computed by an effective procedure (algorithm) can be computed by a Turing machine.
- **Halting problem:** given $\langle M, w \rangle$, decide whether M halts on w. It is undecidable.
- **Post's Correspondence Problem (PCP):** given two lists $A = (w_1 \ldots w_k)$ and $B = (x_1 \ldots x_k)$, is there a sequence $i_1 \ldots i_m$ with $w_{i_1} \ldots w_{i_m} = x_{i_1} \ldots x_{i_m}$? It is undecidable.
- **Modified PCP (MPCP):** the same, but the solution must start with index 1.
- **Primitive recursive function:** built from the zero, successor and projection functions using composition and primitive recursion.

💡 Core concepts

Variants of TM (all equal in power to the basic TM).

- Multi-tape TM: k tapes and k heads. It can be simulated by a single tape with at most quadratic slowdown.
- Nondeterministic TM: several choices per move; simulated by breadth-first search on a deterministic TM.
- Two-way infinite tape, multi-track tape, multi-head, offline TM (read-only input tape).
- TM with stay option (L, R or S).

Techniques for TM construction. Storage in the state (finite control remembers a symbol), multiple tracks (mark symbols with ✓), subroutines (one TM calls another), and shifting over (make space on the tape).

Recursive vs recursively enumerable

Point	Recursive	Recursively enumerable
TM behaviour	halts on all inputs	halts and accepts on strings in L; may loop otherwise
Also called	decidable	semi-decidable (Turing-recognizable)
Complement	always recursive	not always RE
Relation	Recursive ⊂ RE	
Key theorem	if L and its complement are both RE, then L is recursive	

Halting problem is undecidable (proof by contradiction).

1. Suppose a TM H exists: H(⟨M, w⟩) halts with "yes" if M halts on w and "no" otherwise.
2. Build D: on input ⟨M⟩, run H(⟨M, M⟩). If H says "yes", D loops forever; if H says "no", D halts.
3. Run D on ⟨D⟩. If D halts on ⟨D⟩, then H said "no", meaning D does not halt. If D does not halt, then H said "yes", meaning D halts.
4. Either way there is a contradiction, so H cannot exist.

Other undecidable problems. Whether a TM accepts the empty language, whether two CFGs are equivalent, whether a CFG is ambiguous, whether a CFG generates all strings (Σ^*), and PCP. Rice's theorem: every non-trivial property of the language accepted by a TM is undecidable.

Recursive function theory (introduction).

- Initial functions: zero $Z(x) = 0$, successor $S(x) = x + 1$, projection $P_i(x1 \dots xn) = x_i$.
- Primitive recursion: $f(x, 0) = g(x)$ and $f(x, y + 1) = h(x, y, f(x, y))$.
- Example, addition: $add(x, 0) = x$ and $add(x, y + 1) = S(add(x, y))$.
- Example, multiplication: $mul(x, 0) = 0$ and $mul(x, y + 1) = add(mul(x, y), x)$.
- μ-recursive (partial recursive) functions add the minimization operator μ. They equal the Turing-computable functions.
- The Ackermann function is total and computable, but not primitive recursive.

🧮 Solved examples

Example 1 (TM for $L = \{a^n b^n \mid n \geq 1\}$). Idea: replace the leftmost a by X, find the leftmost b and replace it by Y, return, and repeat. When no a is left, check that only Y's remain.

State	a	b	X	Y	B
q0	(q1, X, R)			(q3, Y, R)	
q1	(q1, a, R)	(q2, Y, L)		(q1, Y, R)	
q2	(q2, a, L)		(q0, X, R)	(q2, Y, L)	
q3				(q3, Y, R)	(q4, B, R)
*q4					

Trace for aabb: q0aabb ⊢ Xq1abb ⊢ Xaq1bb ⊢ Xq2aYb ⊢ q2XaYb ⊢ Xq0aYb ⊢ XXq1Yb ⊢ XXYq1b ⊢ XXq2YY ⊢ Xq2XYY ⊢ XXq0YY ⊢ XXYq3Y ⊢ XXYYq3B ⊢ XXYYBq4. Accepted. A blank entry means the TM halts and rejects.

Example 2 (TM for 1's complement of a binary string).

- $\delta(q0, 0) = (q0, 1, R)$
- $\delta(q0, 1) = (q0, 0, R)$
- $\delta(q0, B) = (qf, B, L)$

Input 1011 gives 0100.

Example 3 (TM for unary addition). Input $1^m\ 0\ 1^n$ (e.g. 111011 for 3 + 2). Replace the 0 by 1, move to the end and erase the last 1.

- $\delta(q0, 1) = (q0, 1, R)$, $\delta(q0, 0) = (q1, 1, R)$
- $\delta(q1, 1) = (q1, 1, R)$, $\delta(q1, B) = (q2, B, L)$
- $\delta(q2, 1) = (qf, B, L)$

111011 becomes 11111, which is 5.

Example 4 (PCP instance with a solution).

i	List A	List B
1	1	111
2	10111	10
3	10	0

Sequence 2, 1, 1, 3: A gives 10111 · 1 · 1 · 10 = 101111110 and B gives 10 · 111 · 111 · 0 = 101111110. They match, so this instance has a solution.

Example 5 (PCP instance with no solution). A = (00, 111), B = (0, 11). Each A string is longer than its B string ($|00| > |0|$ and $|111| > |11|$), so for any sequence of indices the A side is strictly longer than the B side. The two strings can never be equal, so this instance has no solution. (Undecidability means no single algorithm works for every instance, not that every instance is hard.)

Key points to remember

- TM 7-tuple: $(Q, \Sigma, \Gamma, \delta, q0, B, F)$, with $\Sigma \subseteq \Gamma$ and $B \in \Gamma - \Sigma$.
- A blank entry in the TM table means "halt and reject".
- Hierarchy of machines: FA < DPDA < NPDA < LBA < TM.
- Recursive languages are closed under complement; RE languages are not.
- PCP is undecidable in general, but a particular instance may be easy to solve.
- Church–Turing thesis is a thesis (cannot be proved), because "effective procedure" is informal.

⭐ Important questions

1. Define a Turing machine. Design a TM for a^nb^n (or $a^nb^nc^n$, or a palindrome) and trace a string.
2. Explain the variants of Turing machines and show they are equivalent to the basic model.

3. Explain the universal TM, LBA and the Church–Turing thesis.
4. Prove that the halting problem is undecidable.
5. Explain PCP and MPCP with an example; distinguish recursive and RE languages; introduce primitive recursive functions.

Last-hour checklist

Read only this in the last hour before the paper.

- ☐ DFA 5-tuple; complete DFA with dead state; divisibility DFA (2r + b) mod k.
- ☐ Subset construction; ε-closure formula for ε-NFA to NFA.
- ☐ Minimization: P0 = {F, Q − F}, split until stable.
- ☐ Moore (n + 1 outputs) vs Mealy (n outputs); conversion both ways.
- ☐ RE identities; Arden's theorem R = QP*.
- ☐ Pumping lemma (regular): |xy| ≤ n, |y| ≥ 1; $a^n b^n$ proof.
- ☐ Ambiguity example id + id * id; Chomsky hierarchy table.
- ☐ Simplification order; CNF steps; GNF form A → aα.
- ☐ PDA 7-tuple; PDA for $a^n b^n$ with ID trace; CFG to PDA rules.
- ☐ Pumping lemma (CFL): |vwx| ≤ n; $a^n b^n c^n$ proof; CFLs not closed under ∩ and complement.
- ☐ TM 7-tuple; TM for $a^n b^n$ with X/Y marking.
- ☐ Halting problem proof; PCP example 2,1,1,3; recursive vs RE.

Books prescribed by AKTU

These are the books named in the official AKTU syllabus. Buy or borrow them through a bookshop or your college library. The WorldCat links show which libraries hold a copy.

#	Title	Author(s)	Publisher	Where to get it legally
1	Introduction to Automata Theory, Languages and Computation (2nd ed.)	J. E. Hopcroft, R. Motwani, J. D. Ullman	Pearson Education Asia	WorldCat
2	Introduction to Languages and the Theory of Computation (3rd ed.)	John C. Martin	Tata McGraw-Hill	WorldCat
3	Elements of the Theory of Computation	H. R. Lewis, C. H. Papadimitriou	PHI	WorldCat
4	Mathematical Foundations of Computer Science	Y. N. Singh	New Age International	WorldCat

Official syllabus: AKTU B.Tech 2nd year CSE syllabus, 2023-24 scheme (PDF)